UNIVERSITÄT BONN

Lab report

Title of Report

modify

Ruben Kemna, Carlos Ridder, Sebastian Sonntag

August 17, 2025

in the Context of the Lab

Efficient Algorithms for selected Topics

of Prof. Dr. Heiko Röglin

Summer Semester 2025

Tutor:

Joshua Könen, Sarah Sturm, Aaron Weinmann

Prof. Dr. Heiko Röglin

Fakultät für Informatik
Universität Bonn

Contents

Remove all `todo`-Commands

1 k-means

Carlos – k-means refers to a clustering algorithm for data points in a vector space $\mathbb{R}^d$. Given a set of n observations $X = \{x_1, \dots, x_n\} \subseteq \mathbb{R}^d$ the task is to find a partition of X into a set of m clusters $S = \{S_1, \dots, S_k\}$ such that the (average) squared euclidean distance between points within each cluster is minimized, i. e. find

$$\arg\min_S \sum_{i=1}^{k} \frac{1}{|S_i|} \sum_{x,y \in S_i} \|x - y\|^2 = \arg\min_S 2 \sum_{i=1}^{k} \sum_{x,y \in S_i} \|x - \mu_i\|^2 \tag{1}$$

where μ_i is the median (center) of cluster S_i, i. e.

$$\mu_i = \frac{1}{|S_i|} \sum_{x \in S_i} x.$$

Equality (1) follows from

$$\begin{aligned}
\sum_{x,y \in S_i} \|x - y\|^2 &= \sum_{x,y \in S_i} \|(x - \mu_i) - (y - \mu_i)\|^2 \\
&= \sum_{x,y \in S_i} \|x - \mu_i\|^2 + \|y - \mu_i\|^2 - 2\langle x - \mu_i, y - \mu_i \rangle \\
&= \sum_{x \in S_i} \|x - \mu_i\|^2 + \sum_{y \in S_i} \|y - \mu_i\|^2 \\
&\quad - 2 \sum_{x,y \in S_i} \langle x - \mu_i, y - \mu_i \rangle
\end{aligned}$$

and

$$\begin{aligned}
\sum_{x,y \in S_i} \langle x - \mu_i, y - \mu_i \rangle &= \left\langle \sum_{x \in S_i} x - \mu_i, \sum_{y \in S_i} y - \mu_i \right\rangle = \langle 0, 0 \rangle \\
&= 0.
\end{aligned}$$

For clustering algorithms like k-means and can be useful to find the closest center point to a given point. For example if there already exists a partition into clusters and a new point gets added it can be greedily extended by adding it to the cluster whose center is closest.

1.1 k-means I

Now in task k-means I we are given a set of one-dimensional points $P \subset \mathbb{R}$, set of center points $C \subset \mathbb{R}$ and a maximum distance $R \in \mathbb{R}_{\geq 0}$ and want to determine for every point $x \in P$ if the distance (with respect to the euclidean distance) to the closest center point is at most R, i. e. if

$$\min_{c \in C} |x - c| \leq R$$

and output the closest center if that is the case and output `none in range` otherwise. This could for example be used as a heuristic to decide if a new clustering should be computed or a new cluster for x should be added.
Because in this case the points are only one-dimensional we can make use of the standard ordering on $\mathbb{R}$. If we first order the center points in ascending order of their value we can use binary search to find the closest center for each point.

```
1: Sort center points C in ascending order.
2: for each point x ∈ P do
3:     Perform binary search on sorted centers to find closest center point c_x ∈ C.
4:     if (x − c_x)^2 > R then
5:         output none in range
6:     else
7:         output c_x
8:     end if
9: end for
```

Algorithm 1.1: closest center in 1d

Theorem 1. *Given a list of n points $P \subset \mathbb{R}$, list of k center points $C \subset \mathbb{R}$ (and a maximum distance $R \in \mathbb{R}_{\geq 0}$) finding the closest center* $\arg\min_{c \in C} |x - c|$ *for each point $x \in P$ (and determining if its distance is at most R) can be done in $\mathcal{O}((k + n) \log k)$.*

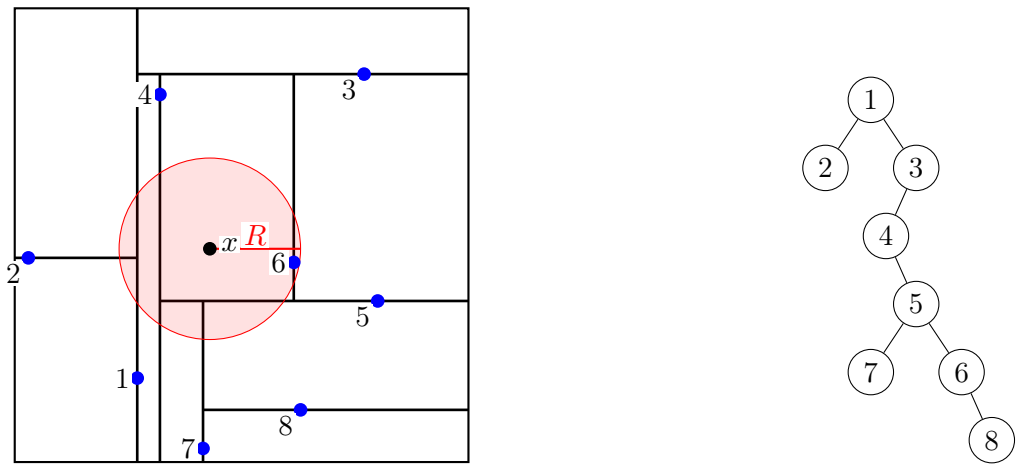
Proof. Sorting the center points can be done in $\mathcal{O}(k \log k)$ using a standard sorting algorithm such as *Merge sort* (Knuth, 1998) and each binary search on the center points takes $\mathcal{O}(\log k)$ time because the search space gets exactly halved in each iteration. Overall this results in a worst-case runtime of $\mathcal{O}(k \log k + n \log k) = \mathcal{O}((k + n) \log k)$. □

1.2 k-means II

Now we are faced with the more realistic setting where $d > 1$. Concretely we are presented with an input stream of points $x_i \in \mathbb{R}^k$ and maximum distances $R_i \in \mathbb{R}_{\geq 0}$. Starting with an empty set of center points C at each step we have to decide if the distance from x_i to the closest center point is at most R_i and if that is not the case, add x_i to C.

A naive approach would be to iterate over all center points in no particular order for each point x_i until the distance is less than R_i or the end is reached. This has a worst-case runtime of i for the i-th point and if every point has to be added to the center points the overall worst-case runtime is $\mathcal{O}(\sum_{i=1}^{n} i) = \mathcal{O}(n^2)$ for an input stream of n points in total.
But this is not fast enough yet, so ideally we want to find a way to decide if the distance to the closest center point is at most R_i in sublinear time by searching through them in a more clever way.

Data structure Building on the idea for k-means I we can construct a data structure for storing the center points that allows for something like binary search in the different dimensions.
This leads to a data structure called $k-d$ trees. $k-d$ trees are binary trees with k-dimensional points at each node together with a $(k-1)$-dimensional hyperplane that splits the space into two half spaces. All points in the left subtree then lie in one half space and all points in the right subtree in the other half space.
The easiest way of selecting the hyperplanes is to select the $(i \mod k)$-th dimension as the *splitting dimension* at the i-th level of the tree and the corresponding coordinate of the point as the *splitting value*.
Inserting a new point then is exactly the same as for standard binary search trees.
Deciding if a point is closer than a given distance to any center point can be done by traversing the tree until a point that is close enough has been found. Let j be the *splitting dimension* and z the *splitting value*, then if $x_j < z$ the left subtree is traversed first and the right subtree only has to be traversed after if $x_j - z \leq R$ which corresponds to the hypersphere with radius R centered at x intersecting the splitting hyperplane.



(a) Partition diagram with query (x, R) (b) Binary tree

Figure 1: A two-dimensional $k-d$ tree constructed by adding 8 random points from the unit cube $[-1, 1]^2$.

Analysis The space complexity of a $k-d$ tree with n nodes is $\mathcal{O}(dn)$.
Inserting a new element into a $k-d$ tree takes $\mathcal{O}(h)$ time, where h is the height of the tree as for any binary tree. It is easy to see that in the worst-case $\mathcal{O}(h) = \mathcal{O}(n)$ but in practice its much better.

Theorem 2. *(Bentley, 1975) After inserting n random points $x_1, \ldots, x_n \in \mathbb{R}^d$ into an empty (d-dimensional) $k-d$ tree, the expected height of the tree is $\mathbb{E}[h] \in \mathcal{O}(\log n)$.*

The runtime of the operation `is_close`(x, R) is somewhat more complex as it not only depends on h but also on which bounding rectangles are intersected by the hypersphere centered at x with radius R and thus potentially have to be searched.
The problem is similiar to nearest-neighbor-search but with early stopping and pruning and for the nearest-neighbor-search in certain well-behaved $k-d$ trees, the following result exists.

Theorem 3. *(Skrodzki, 2019) In a balanced $k-d$ tree the expected runtime of a nearest-neighbor-search is $\mathcal{O}(\log n)$*

The $k-d$ tree in the proof is constructed offline with all of points given by adding them in such an order that the *splitting dimension* always maximizes the spread (biggest distance in that dimension) of the remaining points and the *splitting value* is the median of the remaining points in the *splitting dimension*. This results in a tree with logarithmic depth and bounding rectangles which are closer to squares. However in higher dimensions $d \gg 1$ the pruning becomes less effective because single dimensions contribute less to total distance and it the runtime of nearest-neighbor-search on randomly constructed $k-d$ trees is reported to approach linear time.

Add more concrete results with sources

2 Duckburg

Carlos – In this task we are asked to find an algorithm that maximizes Scrooge McDucks (german: Dagobert Duck) net winnings in the following lottery.
After paying 25$ each, n players choose an integer from $\{1, \ldots, 1000\}$. Then Scrooge is allowed to choose $i \leq 100$ integers. The reward for each participant is now calculated as the minimum distance from his number to any of Scrooges numbers.
So given set of numbers $P \subset \{1, \ldots, 1000\}$ and an integer $k \leq 100$ our task is to find a set S of k integers that minimizes

$$\sum_{x \in P} \min_{s \in S} |x - s|.$$

In other words, we want to select k center points such that the absolute distance to the center of the cluster is minimized, when all points are clustered by assigning them to the closest center point.
This problem is known as *k-center problem*. In general it is *np-hard*

3 Fast and Furious I

SEBASTIAN – We are in an illegal street racing competition in Manhattan and receive a set S of integer coordinates $S = \{c_1, \ldots, c_n\} = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ that we have to visit, then return to the start c_1 to conclude the tour. While our opponent has to make due with a Nissan Skyline, we have a Lamborghini at our disposal that is at least twice as fast. The opponent however will use an optimal tour. As the cars capabilities are known, the victory depends on the length of the tour that we calculate and output as an answer to the coordinates received.

The problem is thus: Find a 2-OPT approximation of the *Traveling Salesman Problem* (TSP) using the L_1 metric and output its length. In the two-dimensional coordinate space $\mathbb{Z}^2$, the distance between any two points $(c_i, c_j) \in S$ with coordinates $(x_i, y_i), (x_j, y_j)$, respectively, is given by

$$\mathrm{d}_1(c_i, c_j) = |x_i - x_j| + |y_i - y_j|. \tag{2}$$

That is, it is the sum of the absolute values of the differences in both coordinates. As Garey et al. (1976) have shown that the L_1TSP is NP-complete, we cannot hope to find any algorithm which runs in polynomial bounded time that finds the optimal tour.

3.1 Nearest Neighbor Algorithm

This very simple heuristic is probably quite close to what most people would use if they were presented with such a task in the real world. In this greedy algorithm, a path is constructed as follows:

1: Start with an arbitrary node.
2: Find node not yet in path that minimizes the distance to the node last added. Add this node to the path.
3: When all nodes are added to the path, connect the node last added and the start node.

Algorithm 3.1: Nearest Neighbor

Clearly, this gives a viable tour, but it cannot guarantee 2-OPT as Rosenkrantz et al. (1977) state in Theorem 4.

Theorem 4. *For a traveling salesman graph with n nodes*

$$\frac{\text{NEARESTNEIGHBOR}}{\text{OPTIMAL}} \leq \frac{1}{2}\lceil \log_2(n) \rceil + \frac{1}{2}\ .$$

Note however that the nearest neighbor algorithm can be programmed to operate in time proportional to n^2, where n is the number of nodes.

Furthermore, randomization of the starting node, i.e. restarting the algorithm multiple times and picking the best run makes the task of finding an instance where this algorithm gives a tour that is larger than $2 \cdot \text{OPT}$ challenging. This is probably exacerbated by the restriction of this exercise to integer coordinates.

3.2 Minimum Spanning Tree

There are certainly better L_1TSP heuristics that guarantee 2-OPT and deliver strong performance in empirical testing (Allison and Noga, 1984). While the following approach, the Minimum Spanning Tree (MST) Heuristic, guarantees 2-OPT, you would not expect it to perform competitively in most cases.

To construct the MST, we can use Prim's algorithm (Prim, 1957). Algorithm 3.2 then runs in time $\mathcal{O}(n^2)$, where n is the number of nodes.

1: Given the coordinates, compute a MST.
2: Duplicate all edges of this MST and return this length.

Algorithm 3.2: Minimum Spanning Tree

Theorem 5. *Algorithm 3.2 gives a 2-approximation of the TSP problem. More precisely, the cost of all edges is* $2 \cdot \text{OPT}_{\text{MST}} \leq 2 \cdot \text{OPT}_{\text{TSP}}$

To prove this theorem, we first prove the following Lemma 1:

Lemma 1. *The cost of an optimal tour* OPT_{TSP} *is at least as large as the weight of a minimum spanning tree* OPT_{MST}.

Proof. Consider the optimal traveling salesman tour OPT_{TSP}. Delete one edge of the TSP tour. This gives a (non-minimal) spanning tree of cost at most OPT_{TSP}. □

From Lemma 1, the proof for Theorem 5 follows immediately.

Proof. We note that this algorithm constructs a valid tour by duplicating all edges of the MST. The cost of this tour is then given by Lemma 1 to be at most $2 \cdot \text{OPT}_{\text{TSP}}$. □

4 Skyline

Sebastian – In this exercise, we are tasked with city planning. We decide to structure the city with a grid and build skyscrapers. In order to ensure a pleasant looking skyline, we are interested in the minimum distance between two skyscrapers of the same height.

The problem is thus: In a grid of dimension n, and given the heights $\in \{0, \ldots, n^2-1\}$ of the n^2 skyscrapers located in the grid, find the minimum distance between two skyscrapers of the same height. The distance metric is defined as the Manhattan Distance (Equation 2) like the problem "Fast and Furious I" from Section 3.

The Algorithm for this problem is fairly simple. We are given the set S of skyscrapers and iterate through them row by row, i.e. $\{s_{1,1}, s_{1,2}, \ldots, s_{1,n}, s_{2,1}, \ldots, s_{n,1}, \ldots, s_{n,n}\}$. The value of the $\{s_{i,j} \mid i, j \in \{1, \ldots, n\}\}$ is the height of the skyscraper and the coordinate is information that is given by its position.

The key point is then that we use the height of the skyscrapers as key and map the coordinates as value into an efficient data structure. Upon insertion of a new skyscraper s, iterate through the other skyscrapers of the same height s' and calculate $\mathrm{d}_1(s, s')$.

While the approach of this Algorithm is correct, it is not efficient enough. The worst-case runtime of this algorithm is $\mathcal{O}(m^2)$ where m is the number of skyscrapers, of which there are n^2 many. For n being the grid dimension, the runtime is consequently $\mathcal{O}(n^4)$.

One optimization is easy to see: As soon as we find a distance that is 1, we immediately return this as an answer, as we will not find a better solution. This reduces the number of skyscrapers that can be of the same height without any of these having distance 1 to at most $n^2/2$. The buildings then form a check pattern, such that the pairwise distance d_1 (Equation 2) between marked cells is ≥ 2.

4.1 Pruning

In order to reduce the number of comparisons, we employ pruning techniques. That is, we examine which comparisons cannot lead to discovering a smaller distance, and can subsequently be skipped.

First, observe that buildings of the same height are inserted in lexicographical order of their coordinates. That means that we can easily iterate through the buildings in (reversed) lexicographical order.

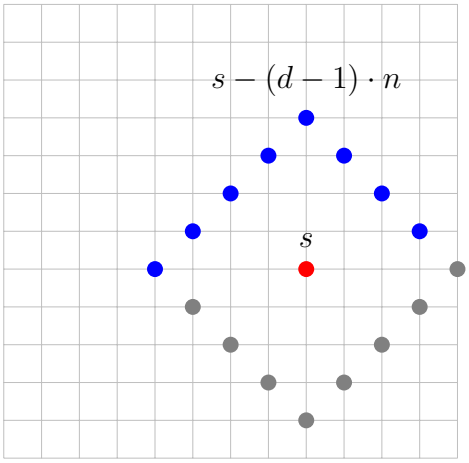
The second observation is that the current smallest distance can serve as a limit to how many other buildings we compute the distance to. This notion is captured in Lemma 2.

Lemma 2. *Define the following enumeration function for grid dimension n:*
$\mathrm{Enum}(i, j) = (i-1) \cdot n + j$*, where i is the y-coordinate and j is the x-coordinate, and $i, j \in \{1, \ldots, n\}$. Let d be the current smallest distance, define a fixed point s by its enumeration, and compare this to points $\{s' \mid s' < s\}$.*

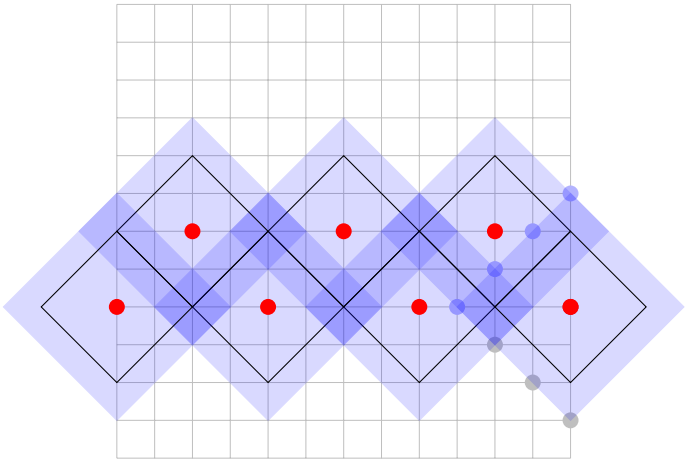
Then for the L_1*-Metric, the smallest number that can yield a distance $< d$ is*

$$s - (d-1) \cdot n, \text{ if this number is } > 0.$$

In any metric space, a sphere is a set of points at a fixed distance, the radius from a specific center point. For the two dimensional Manhattan geometry, the sphere is a



(a) The radius with points p with distance $r = d - 1$ around the center point s in red using the L_1-Metric. Note that the $\{p \mid p < s\}$ are blue, the $\{p \mid p > s\}$ are gray.



(b) A packing of spheres with radius $d - 1$ and L_1-Metric such that no center point (red) is contained by the radius of another point (blue). This is equivalent to center points with radius $d/2$ that cannot intersect (black).

square oriented diagonally to the coordinate axes. Figure 2a shows the set of all points on a square grid that are a fixed distance $r = d - 1$ from the center.

Proof. Enumerate all points row by row and let the radius with distance $r = d - 1$ be defined by the points $\{p \mid \mathrm{d}_1(s, p) = r\}$ around the center point s on a grid of dimension n. Observe that any point outside this radius will have distance at least d.

Then $s - (d - 1) \cdot n$ has exactly distance $d - 1$ from s and is located directly above s (Figure 2a), if this number is > 0, because $(d-1) \cdot n$ is exactly $d-1$ rows with n elements each. Therefore, any number smaller than that will have distance at least d. □

This gives us the stopping conditions for Algorithm 4.1. As soon as we have a smallest distance d, we stop evaluating elements that we know are too far away based on their enumeration. Let s be the last element we add to the list, then for all other pairs of elements $\{s_i, s_j \mid s_i < s_j < s\}$: $\mathrm{d}_1(s_i, s_j) \geq d$, as otherwise d would have been set to a lower value earlier.

This already gives us an upper limit on the number of comparisons we will do: Given d, we will examine at most $d - 1$ rows. Disregarding the distances of elements between different rows, there are at most $\lfloor n/d \rfloor$ elements in each row with pairwise distance $\geq d$. Therefore, over $d - 1$ rows, we have at most $\lfloor n/d \rfloor \cdot (d - 1) \leq \lfloor n - n/d \rfloor \leq n$ elements to compare.

To achieve a better bound on the number of elements, model the interaction between points with L_1-distance $\geq d$. Figure 2b illustrates the following Lemma 3.

```
1: M := data structure that maps heights to skyscrapers.
2: d_1 := given two enumerations of coordinates, return Manhattan distance.
3: distance ← ∞
4: while Set S of skyscrapers not empty AND distance ≥ 2 do
5:     pick next s ∈ S and map enumeration of s to M according to height of s
6:     last ← s − (distance −1) · n
7:     for all other items s_i with same height as s AND s_i ≥ last do
8:         distance ← min(distance, d_1(s, s_i))
9:     end for
10: end while
```

Algorithm 4.1: Skyline with pruning

Lemma 3. *Consider the two dimensional metric space $\mathbb{Z}^2$ and the L_1-metric. Then the following is equivalent:*

(i) Given center points s_i with radii $d-1$ and the condition that no radius can intersect or contain another center point. (blue areas in Figure 2b)

(ii) Given center points s_i with radii $d/2$ and the condition that radii can touch, but not intersect. (black lines in Figure 2b)

Proof. (i) implies that the distance between points is at least $(d-1)+1 = d$, i.e. the halfway point between two points is $\geq d/2$, which is equivalent to (ii). Condition (ii) implies that any two points will have distance at least $2 \cdot d/2 = d$ between them, i.e. no radius of length $d-1$ around any of the center points can contain or intersect another center point, which is equivalent to (i). □

With this result, we can achieve a better bound on the number of elements to examine.

Theorem 6. *In the metric space $\mathbb{Z}^2$ with L_1-metric, and given current distance d between elements, at most $4(nd-n)/d^2$ elements can be in the $d-1$ rows we examine.*

Proof. We interpret the space given by $d-1$ rows as an area where we can place elements s_i with non-overlapping radius $d/2$. The area of $d-1$ rows is $A = n \cdot (d-1)$. The area inscribed by the radius of $d/2$ is $d^2/2$ (two triangles with base d and height $d/2$).

We observe that half of the areas of the radii can be outside A, when the center points of all elements are on the upper or lower row. Thus, the number of s_i in area A is at most: $2 \cdot (n \cdot (d-1))/(d^2/2) = 4(nd-n)/d^2$ □

For $d = 2$, this is maximized and we again get the upper bound of n elements. For bigger d, the bound from Theorem 4.1 on the number of elements to compare is a sizeable improvement in run time. We combine this with the simpler bound from before and get: $\min(\lfloor n - n/d \rfloor\ ,\ 4(nd-n)/d^2) \leq 3n/4$. Still, the worst-case runtime is $\mathcal{O}(n^3)$, as we examine at most n other buildings for every one of the n^2 buildings in the input. Equivalently, the runtime is $\mathcal{O}(m^{3/2})$, where m is the number of skyscrapers in the input.

5 Looking for Oil

RUBEN – In the *Looking for Oil* exercise, we are tasked with selecting a single *axis-aligned rectangle* inside a square field discretized into $n \times n$ parcels. For each parcel we are given an estimated profit (revenue of the oil minus purchasing cost). The task is therefore to choose the rectangle whose total profit is maximized.

5.1 Problem Statement

Input. We are given a square grid of size n with $1 \leq n \leq 300$, with entries $A_{i,j} \in \{-100, \ldots, 100\}$.

Goal. Among all axis-aligned rectangles $R = [i_1..i_2] \times [j_1..j_2]$ with $1 \leq i_1 \leq i_2 \leq n$ and $1 \leq j_1 \leq j_2 \leq n$, maximize

$$\text{val}(R) \;=\; \sum_{i=i_1}^{i_2} \sum_{j=j_1}^{j_2} A_{i,j}.$$

5.2 Maximum Subarray in 1D (Kadane's Algorithm)

Before solving the 2D variant efficiently, we will take a look at the 1D maximum subarray problem. For a sequence $(a_1, \ldots, a_m)$, find a contiguous subarray with maximum sum. Kadane's algorithm maintains

$$\texttt{curEndAt}_i = \max\{a_i,\, \texttt{curEndAt}_{i-1} + a_i\} \quad \text{and} \quad \texttt{best} = \max\{\texttt{best},\, \texttt{curEndAt}_i\}.$$

Intuition: either the optimal subarray ending at i continues the previous one (if it helps) or it restarts at i.

```
1: best ← a_1,  cur ← a_1
2: for i ← 2 to m do
3:     cur ← max(a_i,  cur + a_i)
4:     best ← max(best,  cur)
5: end for
6: return best
```

Algorithm 5.1: Kadane1D$(a_1, \ldots, a_m)$

Correctness (sketch). Every optimal subarray ending at position i either includes position $i-1$ (then extending the best subarray ending at $i-1$ is optimal) or it does not (then it must start at i). Kadane's recurrence enumerates exactly these two cases. The global maximum is the best over all endpoints.

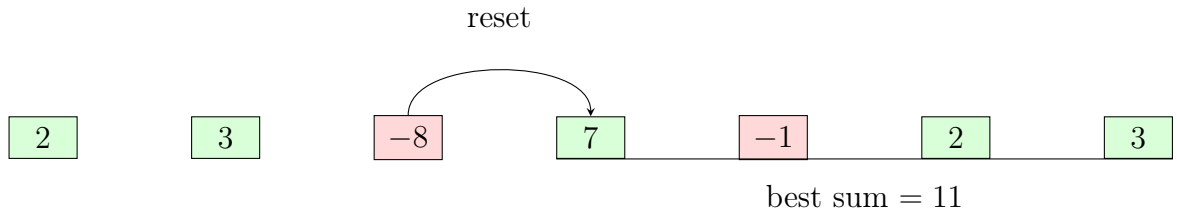


Figure 3: Kadane on $(2, 3, -8, 7, -1, 2, 3)$: the running sum resets after -8, and the best segment is $7, -1, 2, 3$.

Time and space. Kadane runs in $\mathcal{O}(m)$ time and $\mathcal{O}(1)$ extra space.

5.3 From 1D to 2D: Row Compression with Kadane

Any optimal rectangle $[i_1..i_2] \times [j_1..j_2]$ has some top row i_1 and bottom row i_2. For fixed (i_1, i_2) we can *compress* rows i_1 to i_2 into a single 1D array of column sums

$$b_j = \sum_{i=i_1}^{i_2} A_{i,j} \quad (1 \le j \le n).$$

Then the best horizontal span $[j_1..j_2]$ for these two rows is exactly the maximum subarray of $(b_1, \ldots, b_n)$, i.e. returned by Kadane1D. Enumerating all $i_1 \le i_2$ gives the global optimum.

```
1: best ← −∞
2: for topRow ← 1 to n do
3:     for all j: col[j] ← 0                          ▷ start a new strip at topRow
4:     for bottomRow ← topRow to n do
5:         for j ← 1 to n do
6:             col[j] ← col[j] + A_{bottomRow, j}      ▷ add the new row elements
7:         end for
8:         best ← max(best, Kadane1D(col))
9:     end for
10: end for
11: return best
```

Algorithm 5.2: MaxProfitRectangle2D($A \in \mathbb{Z}^{n \times n}$) via row compression

Correctness (sketch). Fix any top/bottom rows (i_1, i_2). For these rows, the sum of a rectangle equals the sum of its columns in the compressed array. Hence the best left/right boundary is exactly the maximum subarray. Taking the max over all (i_1, i_2) covers the optimal rectangle.

5.4 Time and Space Complexity

The outer double loop enumerates $\mathcal{O}(n^2)$ row pairs, each Kadane call costs $\mathcal{O}(n)$ time, and the compression update per bottom row is also $\mathcal{O}(n)$. Therefore the total running time is

$$\mathcal{O}(n^2) \cdot \mathcal{O}(n) \;=\; \mathcal{O}(n^3).$$

The space needed to exectute the algorithm is only $\mathcal{O}(n)$ for `col` (besides the input matrix).

6 Sweet Tooth

Ruben – Next we will turn to sweets distribution amongst children. In this exercise, we must split a stash of sweets among c children so that everyone gets the same number of sweets. Each child has preferences per sweet type (smaller numbers mean "more preferred"). We may eat the leftovers. The task is to find the smallest priority threshold p such that a fair distribution exists where each child only receives sweets of priority at most p.

6.1 Problem Statement

Let there be t sweet types and a count vector $a \in \mathbb{N}^t$ with total $s = \sum_{k=1}^{t} a_k$. There are c children and a preference matrix $P \in \{1, \ldots, P_{\max}\}^{c \times t}$ with entries $P_{i,k}$ (lower is better). Each child must receive exactly

$$\text{share} \;=\; \left\lfloor \tfrac{s}{c} \right\rfloor$$

sweets; the remaining $s - c \cdot \text{share}$ pieces are ours.

For a threshold $p \in \{1, \ldots, P_{\max}\}$, define decision variables $x_{i,k} \in \mathbb{N}$: number of sweets of type k given to child i.

$$\begin{aligned}
&\sum_{i=1}^{c} x_{i,k} \;\le\; a_k && \forall k = 1, \ldots, t \quad \text{(limited stock)} \\
&\sum_{k=1}^{t} x_{i,k} \;=\; \text{share} && \forall i = 1, \ldots, c \quad \text{(fair share)} \\
&x_{i,k} \;=\; 0 && \text{if } P_{i,k} > p \quad \text{(respect threshold)} \\
&x_{i,k} \in \mathbb{Z}_{\ge 0} && \text{(integrality)}.
\end{aligned}$$

We seek the minimum p for which a feasible solution exists.

6.2 Flow Modeling

Fix a threshold p. We decide feasibility at this p by building a three-layer flow network

$$s \longrightarrow \text{types } (T_1, \ldots, T_t) \longrightarrow \text{children } (C_1, \ldots, C_c) \longrightarrow t.$$

The edges and capacities encode stock limits, the preference cutoff p, and each child's quota *share*:

- $s \to T_k$ has capacity a_k: at most a_k items of type k can be allocated in total (stock).
- $T_k \to C_i$ exists *iff* $P_{i,k} \leq p$. For its capacity we can safely use

  $$\text{cap}(T_k \to C_i) \;=\; \min\{a_k,\ \textit{share}\}.$$

  This is the minimal safe capacity. Any larger value is also feasible, but redundant because the flow on $T_k \to C_i$ is already bounded by $s \to T_k$ (at most a_k) and by $C_i \to t$ (at most *share*). Any smaller value can exclude feasible allocations when a child must receive (almost) all of its *share* from type k.
- $C_i \to t$ has capacity *share*: this enforces that each child receives exactly *share* sweets once the max flow hits the target value (see correctness).

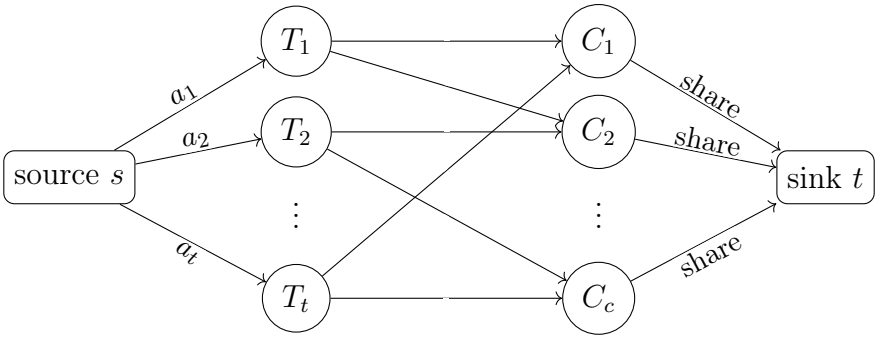


Figure 4: Three-layer max-flow network for threshold p: source→types (stock a_k), types→children (only present if $P_{i,k} \leq p$), children→sink (quota *share*).

We only insert edges $T_k \to C_i$ that satisfy the preference cutoff $P_{i,k} \leq p$; disallowed pairs are absent from the network. With this construction, a feasible allocation at threshold p exists iff the maximum s-t flow equals $c \cdot \textit{share}$.

After now having found a way to test whether there exists a feasible allocation for a fixed threshold p, we search for the *minimum* feasible threshold by binary search over

$$p \in \{1, \ldots, P_{\max}\}, \qquad P_{\max} := \max_{i,k} P_{i,k}$$

(which equals t if preferences are ranks $1, \ldots, t$).

6.3 How We Compute the Max Flow

Any max-flow/min-cut algorithm would decide feasibility at a fixed threshold p (e.g., Edmonds and Karp (1972), Dinitz (1970), or Push–Relabel by Goldberg and Tarjan (1988)). We implement *Dinitz's algorithm* because it fits layered networks well and is simple with adjacency lists.

Dinitz (three-step view). We work on the *residual network* (forward edges with residual $c_f > 0$ and reverse edges that allow canceling flow).

Step 1 (BFS / level graph). Run a BFS from s to label levels $\ell(\cdot)$ using only residual edges. Keep only *forward* residual edges with $\ell(v) = \ell(u) + 1$; this acyclic subgraph is the *level graph*.

Step 2 (early stop). If t was not reached during BFS, there is no augmenting path; return the current flow as *maximum*.

Step 3 (DFS / blocking flow). Using only edges of the level graph, do multiple DFS from s to t. Each DFS finds a level-respecting augmenting path; push the path's bottleneck value and update residuals. Maintain a *current-arc* per node to skip exhausted edges. Stop when a *blocking flow* is reached (no more s-t paths in the level graph).

Repeat Steps 1–3 until Step 2 triggers. With integer capacities, Dinitz returns an *integral* max flow, which we directly read as integer assignments $x_{i,k}$ on the $T_k \to C_i$ edges.

Complexity. The standard worst-case bound is $O(EV^2)$ per feasibility check (here $V = 2+t+c$, $E \le t+c+ct$). Our network is shallow (paths have three naturally arising levels), so it's expected to perform well in practice.

6.4 Correctness

Theorem 7. *For any threshold p, there exists a feasible allocation $\{x_{i,k}\}$ that satisfies stock, quota, and threshold constraints if and only if the maximum flow equals $c \cdot share$ in the network defined above.*

Proof sketch. ($\Rightarrow$) Given a feasible $\{x_{i,k}\}$, send $x_{i,k}$ units along source$\to$type k $\to$child i $\to$sink. All capacities are respected by construction; the total flow is $\sum_i \text{share} = c \cdot \text{share}$.

($\Leftarrow$) From a max flow of value $c \cdot \text{share}$, read off $x_{i,k}$ from the flow on edges type $k \to$ child i. Capacities ensure $\sum_i x_{i,k} \le a_k$ and $\sum_k x_{i,k} \le \text{share}$; the child$\to$sink edge saturating to share forces equality. Missing edges enforce $x_{i,k} = 0$ when $P_{i,k} > p$. □

6.5 Time and Space Complexity

Time. Let $V = 2 + t + c$ and $E \le t + c + ct$ (source→t types, up to ct type→child edges, c child→sink edges). One max-flow check with Dinitz runs in worst-case

$$\mathcal{O}(E\,V^2) \;=\; \mathcal{O}\big((ct)\,(c{+}t)^2\big).$$

Binary search over $p \in \{1, \dots, P_{\max}\}$ adds a $\log P_{\max}$ factor:

$$\mathcal{O}\big(\log P_{\max} \cdot (ct)\,(c{+}t)^2\big)\,.$$

Space. Storing the residual network and Dinitz's working arrays uses $\mathcal{O}(V + E) = \mathcal{O}(ct + c + t) = \mathcal{O}(ct)$ space per feasibility check.

References

D. C. Allison and M. Noga. The l1 traveling salesman problem. *Information Processing Letters*, 18(4):195–199, 1984. ISSN 0020-0190. doi:https://doi.org/10.1016/0020-0190(84)90110-8. URL https://www.sciencedirect.com/science/article/pii/0020019084901108.

J. L. Bentley. Multidimensional binary search trees used for associative searching. *Commun. ACM*, 18(9):509–517, Sept. 1975. ISSN 0001-0782. doi:10.1145/361002.361007. URL https://doi.org/10.1145/361002.361007.

Y. A. Dinitz. Algorithm for solution of a problem of maximum flow in a network with power estimation. *Soviet Mathematics Doklady*, 11(5):1277–1280, 1970. URL https://www.mathnet.ru/eng/dan35701. English translation of Doklady Akademii Nauk SSSR, 194(4):754–757 (1970).

J. Edmonds and R. M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972. doi:10.1145/321694.321699. URL https://doi.org/10.1145/321694.321699.

M. R. Garey, R. L. Graham, and D. S. Johnson. Some np-complete geometric problems. In *Proceedings of the Eighth Annual ACM Symposium on Theory of Computing*, STOC '76, page 10–22, New York, NY, USA, 1976. Association for Computing Machinery. ISBN 9781450374149. doi:10.1145/800113.803626. URL https://doi.org/10.1145/800113.803626.

A. V. Goldberg and R. E. Tarjan. A new approach to the maximum-flow problem. *Journal of the ACM*, 35(4):921–940, 1988. doi:10.1145/48014.61051. URL https://doi.org/10.1145/48014.61051.

D. E. Knuth. *Section 5.2.4: Sorting by Merging*, volume 3 of *The Art of Computer Programming*. Addison-Wesley, 2 edition, 1998. ISBN 0-201-89685-0.

R. C. Prim. Shortest connection networks and some generalizations. *The Bell System Technical Journal*, 36(6):1389–1401, 1957. doi:10.1002/j.1538-7305.1957.tb01515.x.

D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis, II. An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3):563–581, 1977. doi:10.1137/0206041. URL https://doi.org/10.1137/0206041.

M. Skrodzki. The k-d tree data structure and a proof for neighborhood computation in expected logarithmic time, 2019. URL https://arxiv.org/abs/1903.04936.